

Table 3: Overall results of GPTScan’s accuracy evaluation.

Dataset Name	TP	TN	FP	FN	Sum
Top200	0	283	13	0	296
Web3Bugs	40	154	30	8	232
DefiHacks	10	19	1	4	34

RQ5: Can GPTScan discover new vulnerabilities that were previously missed by human auditors?

5.1 RQ1: Measuring False Positives in the Non-vulnerable Top Contracts

In RQ1, we aim to measure GPTScan’s false alarm rate in analyzing non-vulnerable contracts. This is important because when using GPTScan for massive scanning of on-chain token contracts, we want to minimize the false alarms that require manual checking.

For this purpose, we have collected the *Top200* dataset, which consists of 303 contract projects that are deemed non-vulnerable. We present GPTScan’s analysis result of *Top200* in Table 3. Along with the results of *Web3Bugs* and *DefiHacks*, we calculate the accuracy metrics at the function level for each tested vulnerability type. For example, if a project has been tested with five vulnerability types, the sum of all true positives, false positives, true negatives, and false negatives for this project should be 5. More specifically, **TP** is the number of true positives. One true positive is counted when GPTScan successfully detects a ground-truth vulnerable function for the tested vulnerability type.

TN is the number of true negatives. One true negative is counted when GPTScan correctly does not report any vulnerable function for the tested vulnerability type.

FP is the number of false positives. One false positive is counted when GPTScan incorrectly reports one or more vulnerable functions for the tested vulnerability type that has no corresponding ground-truth vulnerabilities in the tested project.

FN is the number of false negatives. One false negative is counted when GPTScan fails to detect the ground-truth vulnerable function for the tested vulnerability type.

Based on the calculation of these metrics, GPTScan reports 13 FPs and 283 TNs for the *Top200* dataset, as shown in Table 3. As a result, the false positive rate of GPTScan in analyzing non-vulnerable top contracts like *Top200* is 4.39%. Moreover, we find that GPTScan has a similar precision when analyzing *Top200* and *DefiHacks*, both of which are token contracts with around 2 Solidity files per project (see Table 2). When analyzing large projects like those in *Web3Bugs*, the precision drops from around 90% (90.91% for *DefiHacks*) to 60% (57.14% for *Web3Bugs*). The drop in precision is likely because the smart contract code in *Web3Bugs* is more diverse, given that *Web3Bugs* contains an average of 36 Solidity files per project (see Table 2). In contrast, smart contracts in *DefiHacks* and *Top200* mainly implement common token functionalities using an average of 2 Solidity files per project, potentially triggering only a limited set of false positives in GPTScan. In §5.2, we will further discuss the root causes of GPTScan’s false positives.

Answer for RQ1: GPTScan achieves a low false positive rate of 4.39% when analyzing non-vulnerable top contracts like *Top200*. It also demonstrates similar performance in analyzing *DefiHacks*, with a precision of 90.91%. These results indicate that GPTScan

is suitable for massive scanning of on-chain token contracts. Moreover, when analyzing large contract projects in *Web3Bugs*, GPTScan still achieves an acceptable precision of 57.14%.

5.2 RQ2: Efficacy for Detecting Vulnerable Contracts

In RQ2, we assess the effectiveness of GPTScan in analyzing vulnerable contracts in the *Web3Bugs* and *DefiHacks* datasets, and compare its effectiveness with existing tools.

As shown in Table 2, the *Web3Bugs* dataset contains 48 ground-truth logic vulnerabilities, while the *DefiHacks* dataset has 14. Table 3 presents the scanning results of these two datasets using GPTScan. In the case of *Web3Bugs*, GPTScan analyzed a total of 232 vulnerability types across 72 projects, detecting 40 TPs and missing 8 FNs, while incurring 30 FPs. Consequently, GPTScan achieved a recall of 83.33% and an F1 score of 67.8% on this dataset. For *DefiHacks*, GPTScan analyzed a total of 34 vulnerability types across 13 projects, detecting 10 TPs and missing 4 FNs, while incurring 1 FP. On this dataset, GPTScan’s recall is 71.43% and the F1 score is 80%. These results demonstrate that GPTScan effectively detects vulnerable contracts for the covered logic vulnerability types. Following the initial precision analysis in §5.1, we now analyze the root causes of GPTScan’s false negatives and false positives.

In the 12 false negative cases, 4 of them are *Price Manipulation by AMM* and 3 of them are *Risky First Deposit*. The main reason for these two kinds of false negatives is that GPTScan does not implement an alias analysis in the static check, causing failure during static dataflow tracing. Additionally, there are 2 cases of *Front Running*, where the scenarios or properties are not accurately matched by GPT. Furthermore, there are 2 cases of *Slippage* and 1 case of *Unauthorized Transfer*. Similar to the false positive cases, The main reason for the false negative *Slippage* cases is the existence of numerous variants of slippage checks, making them challenging to detect using GPT and static analysis. In the case of *Unauthorized Transfer*, the main reason for this false negative is that GPT failed to distinguish the inconsistency between the comment and code.

GPTScan achieves effective vulnerability detection above at an acceptable false alarm rate. Among the 44 false positive cases from the three datasets, 15 (34.09%) were related to *Price Manipulation by AMM*, followed by 11 (25.00%) cases of *Unauthorized Transfer*. For these two types, the main reason for the false alarms is that these vulnerabilities require specific triggering conditions involving other related logic, which may not be contained within a single function and its callers or callees. For example, in *Unauthorized Transfer*, the checks for the allowance/approval from the address owner can occur at various positions in the logic chain and may involve multiple functions. Similarly, the function that calculates the price with AMM for *Price Manipulation* may not be used by other functions responsible for swapping or buying tokens, leading to the vulnerabilities not being triggered in those circumstances.

Additionally, there were 5 cases of *Risky First Deposit* and 5 cases of *Slippage*. For *Risky First Deposit*, the false alarms occurred because there were many statements related to checking the supply and setting the share, making it challenging for GPT to understand lengthy code segments accurately. Regarding *Slippage*, the false

alarms were mainly due to two factors. First, similar to *Unauthorized Transfer*, the check for slippage can happen at any position in the logic chain, and second, slippage checks can take many different forms and variants, making them difficult to detect with GPT and static analysis. For this vulnerability type, our focus was on achieving a higher recall at the cost of slightly sacrificing precision. There were also 4 cases of *Wrong Interest Rate Order*, 3 cases of *Approval Not Cleared*, and 1 case of *Wrong Checkpoint Order*. For *Wrong Interest Rate Order* and *Wrong Checkpoint Order*, these vulnerabilities are intricately related to the business logic of the project itself, making it challenging to reduce false alarms without comprehensive knowledge of the project’s design. As for *Approval Not Cleared*, the false alarms were primarily because the function may not always be used to transfer tokens, causing GPTScan to detect it erroneously.

Comparison with existing tools. While there are many specific static analysis tools (e.g., [28, 29, 47, 56]), they almost do not cover any of the logic vulnerabilities targeted in this paper. We thus selected two comprehensive vulnerability detection tools, one open-source tool, Slither [37], and MetaScan’s online static scanning service [19, 23], referred to as MScan. Both tools have over a hundred vulnerability detection rules, but the rules related to GPTScan are *unchecked-transfer*, *arbitrary-send-eth*, and *arbitrary-send-erc20* for Slither (corresponding to *Unauthorized Transfer* in GPTScan), and two *Price Manipulation* vulnerabilities for MScan.

We ran Slither on all three datasets and found a total of 13,144 warnings. Among these, only 101 of *unchecked-transfer*, 23 of *arbitrary-send-eth*, and 22 of *arbitrary-send-erc20* are related to the *Unauthorized Transfer* vulnerability in GPTScan. Unfortunately, all of them were false positives after careful manual checking. There are mainly two reasons for this. Firstly, Slither does not correlate call chain information. Many false positive cases involve internal or private functions that have already been checked for unauthorized transfer when they are called. In GPTScan, we analyze the current function and its caller together, effectively addressing the issue of missing contextual semantics. Secondly, Slither is unable to correctly detect variants of transfer behavior in *Unauthorized Transfer*, such as burning tokens, leading to its inability to detect vulnerabilities in the dataset. GPTScan relies on GPT to gain the ability to analyze code semantics, which, when combined with code context and calling relationships, can more accurately address these problems.

We also ran MScan on the *DefiHacks* dataset, as 12 of the total 14 vulnerabilities in this dataset are related to *Price Manipulation*. Among these 12 true *Price Manipulation* vulnerabilities, MScan detected 7, achieving a recall of 58.33% and a precision of 100% for *Price Manipulation*. However, MScan failed to detect any other type of logic vulnerabilities. MScan achieved high precision because it used some attack incidents in the *DefiHacks* dataset to derive hard-coded patterns for *Price Manipulation*, including the matching of specific function and variable names. However, in cases where hard-coded patterns are not applicable, MScan cannot generalize to detect variants of *Price Manipulation* vulnerabilities.

For GPT-based tools, the only available study at the time of our submission was conducted by David et al. [34]. Unfortunately, they did not release their tool, and there was insufficient information for us to reproduce it. Therefore, we rely on the statistics provided in

Table 4: Raw functions before and after static confirmation.

Vulnerability Type	Before	After
Approval Not Cleared	34	12
Risky First Deposit	100	21
Price Manipulation by AMM	187	114
Price Manipulation by Buying Tokens	8	8
Vote Manipulation by Flashloan	2	0
Front Running	6	4
Wrong Interest Rate Order	150	11
Wrong Checkpoint Order	49	1
Slippage	99	42
Unauthorized Transfer	12	8
Total	647	221

their paper for comparison. According to the paper, their pure GPT-based approach achieved a precision of 4.14%, a recall of 43.84%, and an F1 score of 7.57% with the GPT-4-32k model, and a precision of 4.30%, a recall of 35.62%, and an F1 score of 7.68% with the Claude-v1.3-100k model, respectively. The false positives are significantly higher than those of GPTScan, mainly because their tool did not validate the GPT output as GPTScan does in §4.4, and thus could be more easily affected by GPT’s inherent problems like hallucination [64], bias in training data, and ambiguity in questions. Indeed, RQ3 in §5.3 suggests a similar finding by measuring the GPT-only result in GPTScan (see details in Table 4).

Answer for RQ2: GPTScan shows its efficacy in detecting ground-truth logic vulnerabilities in the *Web3Bugs* and *DefiHacks* datasets, with a recall of 83.33% and an F1 score of 67.8% for *Web3Bugs*, and a recall of 71.43% and an F1 score of 80% for *DefiHacks*, better than existing static and GPT-based tools.

5.3 RQ3: Effectiveness of Static Confirmation

In RQ3, we conduct a further analysis of GPTScan’s intermediate results on *Web3Bugs* to examine how static confirmation reduces false positives generated by pure GPT-based matching.

Table 4 shows the raw functions reported by GPTScan before and after static confirmation. Note that one vulnerability type may have multiple functions (the final result counts either TP or FP once, according to the calculation in §5.1), and these functions are not merged yet (i.e., a function A and the combination of function A and all its callers would be counted multiple times) that will be done in the final result. Hence, so the number of “after” cases shown here is much larger than the final TP+FP in Table 3. From the result, we observe that static confirmation effectively filters out most false positive cases for the vulnerability types: *Wrong Interest Rate Order*, *Wrong Checkpoint Order* and *Risky First Deposit*. The reason is that the description of scenarios and properties for these three types is coarse-grained, leading to many candidate functions passing the GPT-based matching step. In static confirmation, GPTScan can further instruct GPT to identify related statements and variables, filtering out those that do not satisfy the vulnerability types. Overall, after static confirmation, only 221 raw functions remain out of the original 647 functions. This indicates that static confirmation successfully filters out two-thirds of the false positives.

We further analyze the negative impact of static confirmation. Among the 426 cases filtered out, only 3 ground-truth cases were

Table 5: Running time and financial costs of GPTScan.

Dataset	KL*	T**	C***	T/KL	C/KL
Top200	134.32	1,437.37	0.7507	10.70	0.005589
Web3Bugs	319.88	4,980.57	3.9682	15.57	0.018658
DefiHacks	17.82	375.41	0.2727	21.06	0.015303
Overall	472.02	6,793.35	4.9984	14.39	0.010589

* KL for KLoC; ** T for Time; *** C for Financial Cost.

initially matched by GPT but later excluded by static analysis, resulting in 3 false negatives. Another false negative was related to compilation problems. The remaining four did not pass the GPT-based scenario and property matching step. This indicates that static confirmation has only a minor impact on the false negatives.

Answer for RQ3: Static confirmation effectively filtered out 65.84% of the false positive cases in the *Web3Bugs* dataset, while having only a minor impact on the false negative cases.

5.4 RQ4: Performance and Financial Overhead

In RQ4, we evaluate the running time and financial costs of GPTScan when using OpenAI’s GPT-3.5-turbo API. We considered only the costs associated with interacting with GPT and conducting static analysis. We measured the time and financial cost of GPTScan on all three datasets, and the results are shown in Table 5. In this experiment, we used tiktoken [20], a tokenization tool published by OpenAI and used for GPT models, to estimate the number of tokens sent and received by GPTScan. With the number of tokens sent and received, we can estimate the financial cost of GPTScan. The total number of lines of code is 472K, and it took 6,793.35 seconds and 4.9984 USD to complete the scan. On average, it takes 14.39 seconds and 0.010589 USD to scan per thousand lines of code.

On *Top200*, the scan cost per thousand lines of code is the cheapest, and the scan speed per thousand lines of code is the fastest. This is because most candidate functions are filtered out in GPTScan’s first two steps, without the need for finding related variables and expressions for static check. On *Web3Bugs* and *DefiHacks*, the scan cost per thousand lines of code is the most expensive and the scan speed per thousand lines of code is the slowest, respectively. Projects in *Web3Bugs* and *DefiHacks* are more complex than *Top200*, and there are more complex candidate functions to be scanned. These complex functions could not be filtered by static filtering and scenario matching, which causes more time and financial cost.

Answer for RQ4: GPTScan is fast and cost-effective, taking an average of only 14.39 seconds and 0.01 USD to scan per thousand lines of Solidity code in the tested datasets. The relatively higher cost and slower speed for *Web3Bugs* and *DefiHacks* can be attributed to the presence of more complex functions that cannot be filtered out by static filtering and scenario matching.

5.5 RQ5: Newly Discovered Vulnerabilities

In RQ5, we perform a thorough analysis of GPTScan’s results on the *Web3Bugs* dataset to see if it could identify new vulnerabilities that were previously missed by human auditors. Interestingly, GPTScan successfully discovered 9 vulnerabilities from 3 different types, which did not appear in the audit reports of Code4rena. Among these 9 newly discovered vulnerabilities, 5 are *Risky First Deposit*,

```

1 function deposit(uint _amount) external {
2     ...
3     uint _pool = balance();
4     uint _totalSupply = totalSupply();
5     if (_totalSupply == 0 && _pool > 0) { // trading fee
6         accumulated while there were no IF LPs
7         vusd.safeTransfer(governance, _pool);
8         _pool = 0;
9     }
10    uint shares = 0;
11    if (_pool == 0) {
12        shares = _amount;
13    } else {
14        shares = _amount * _totalSupply / _pool;
15    }
16 }

```

Figure 6: Risky First Deposit in 2022-02-hubble.

```

1 function pendingRewards(uint256 _pid, address _user) external
2     view returns (uint256) {
3     PoolInfo storage pool = poolInfo[_pid];
4     UserInfo storage user = userInfo[_pid][_user];
5     uint256 accRewardsPerShare = pool.accRewardsPerShare;
6     uint256 lpSupply = pool.lpToken.balanceOf(address(this));
7     if (block.number > pool.lastRewardBlock && lpSupply != 0) {
8         uint256 multiplier = getMultiplier(pool.lastRewardBlock
9             , block.number);
10        uint256 rewardsAccum = multiplier.mul(rewardsPerBlock).
11            mul(pool.allocPoint).div(totalAllocPoint);
12        accRewardsPerShare = accRewardsPerShare.add(
13            rewardsAccum.mul(1e12).div(lpSupply));
14    }
15    return user.amount.mul(accRewardsPerShare).div(1e12).sub(
16        user.rewardDebt);
17 }

```

Figure 7: Price Manipulation by AMM in 2021-09-sushimiso.

3 are *Price Manipulation by AMM*, and 1 is *Front Running*. In the following paragraphs, we present one example of each type of newly discovered vulnerability for further discussion.

Risky First Deposit. Among the newly discovered vulnerabilities, 56% of them are *Risky First Deposit*. In the example shown in Figure 6, on line 11, when the variable `_pool` is 0, indicating an empty liquidity pool, the depositor can obtain all the shares from the pool. The presence of both `_totalSupply` and `_pool` variables to represent the liquidity amount in the pool may confuse human auditors. Although lines 5 to 8 properly handle the case when `_totalSupply` is 0, this specific condition involving `_pool` on line 11 creates a vulnerability that could be missed.

Price Manipulation by AMM. Another 33% of the newly discovered vulnerabilities are *Price Manipulation by AMM*. In the example shown in Figure 7, the `pendingRewards` function is used to calculate the rewards that can be claimed by the user. On line 9, when the pool is not empty, the amount of rewards that can be redeemed by the user is calculated based on the total supply in the pool. However, the total supply can be controlled by users, allowing them to manipulate the redeemed amount and exploit the contracts.

Front Running. There is one case of *Front Running* shown in Figure 8, in which the token to be minted should be previously transferred (line 1). However, anyone can call the `mint` function to mint tokens that are transferred but not minted, as there is only a check with the cached amount of the contract (line 7), but not the cached amount of a specific user. This vulnerability allows an attacker to front run the minting process. When a user has

```

1  /// @notice The lp tokens that the user contributes need to
    have been transferred previously, using a batchable
    router.
2  function mint(address to)
3      public
4      beforeMaturity
5      returns (uint256 minted)
6  {
7      uint256 deposit = pool.balanceOf(address(this)) - cached;
8      minted = _totalSupply * deposit / cached;
9      cached += deposit;
10     _mint(to, minted);
11 }

```

Figure 8: Front Running in 2021-08-yield.

transferred a token but not minted it, the attacker could front run the mint function to mint the token before the legitimate user.

Answer for RQ5: GPTScan identified 9 new vulnerabilities not present in the audit reports of Code4rena. This highlights the value of GPTScan as a useful supplement to human auditors.

6 DISCUSSION

In this section, we discuss the current limitations in GPTScan and the potential use of employing other GPT models.

Current limitations in design and implementation. In §4.3, the modifiers filtering part only utilized a whitelist to filter the modifiers with access control. However, this filtering method can lead to false positives or negatives of vulnerabilities. To enhance accuracy, a more precise approach is required, which involves retrieving the definition of modifiers and conducting a detailed semantic analysis on them. For the static analysis part in §4.4, a simple method was used to analyze the control flow graph and data dependence graph. This analysis is not path-sensitive, meaning that some path-related issues, such as the reachability of certain execution paths under specific conditions, might be overlooked. It could be improved by introducing symbolic execution engines to the static analysis part.

The use of other GPT models and parameters. As mentioned in §4.5, GPTScan employs the widely used GPT-3.5-turbo model [27] as its GPT model. We also conducted a preliminary test using GPT-4, but we did not observe a notable improvement, while the cost increased 20 times. This finding suggests that GPTScan does not necessarily require more powerful GPT models. As the temperature parameter is set to zero, the answers of the GPT model tend to be deterministic. A higher temperature might lead to more creative answers, but it could also result in more false positives or false negatives. However, reproducing results becomes more challenging with a higher temperature. In the future, we plan to conduct a systematic test of various GPT models within the context of GPTScan, including Google Bard, Claude (when we have API access to them), and the self-trained LLaMA model, as well as the influence of different parameters on GPTScan.

7 RELATED WORK

In this section, we discuss some related work. Various research and tools have focused on vulnerability detection in smart contracts. Traditional static analysis tools, such as Slither [37], Vandal [30], Ethainter [29], Zues [43], and Securify [56], are used to analyze the source code and detect vulnerabilities. Symbolic execution tools like Manticore [47] and Mythril [13] can perform bound checks and

detect vulnerabilities in bytecode and source code. These analysis tools have been applied to detect vulnerabilities in smart contracts, such as re-entrancy [52, 61], arithmetic overflow [54], state inconsistency problems [28], and access control problems [36, 39, 46]. Dynamic analysis tools, such as fuzz testing [40, 41, 59, 63], automatically generate test cases or inputs for smart contracts to find abnormal behaviors during runtime. Formal verification techniques like Verx [51] and VeriSmart [53] can be used to check user-provided specifications. Nevertheless, Zhang et al. [65] suggested that more than 80% of exploitable bugs are machine undetectable.

Before the advent of ChatGPT (GPT-3.5) [49], most NLP-based vulnerability detection methods [32, 33, 48, 55, 58] involved feeding code into binary or multi-classification models. Now, with the development of instructing GPT [57] and other research providing few-shot learning capabilities [31], interactive solutions can be used for tasks like code repair [42, 60] and vulnerability detection [34]. However, according to the research by David et al. [34], the GPT-4 model itself cannot accurately detect vulnerabilities. Chen et al. [38] fine-tuned the GPT-3 model for improved performance in GUI graphical interface testing tasks and utilized it for automated testing of Android applications. Additionally, PentestGPT [24] and ChatRepair [60] utilized feedback from the execution results to enhance the performance of the GPT model during interactions.

8 CONCLUSION

In this paper, we proposed GPTScan, the first tool combining GPT with static analysis for smart contract logic vulnerability detection. GPTScan utilized GPT to match candidate vulnerable functions based on code-level scenarios and properties, and further instructed GPT to intelligently recognize key variables and statements, which were then validated by static confirmation. Our evaluation on three diverse datasets with around 400 contract projects and 3K Solidity files showed that GPTScan achieves high precision (over 90%) for token contracts and acceptable precision (57.14%) for large projects, as well as a recall of over 70% for detecting ground-truth logic vulnerabilities. GPTScan is fast, cost-effective, and capable of discovering new vulnerabilities missed by human auditors. In future work, we will expand GPTScan's support for more logic vulnerability types.

ACKNOWLEDGEMENTS

We thank Dawei Zhou, Zhe Wang, Guorui Fan, Liwei Tan, Hao Zhang, and other colleagues at MetaTrust Labs for their help with GPTScan, as well as anonymous reviewers for their constructive feedback. This research/project is supported by the National Research Foundation Singapore and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-RP-2020-019), the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

REFERENCES

- [1] 2016. <https://www.coindesk.com/learn/understanding-the-dao-attack/>
- [2] 2021. <https://github.com/code-423n4/2021-11-yaxis>
- [3] 2022. <https://www.freecodecamp.org/news/what-is-yaml-the-yml-file-format/>